

Generative AI in Cybersecurity

Retrofit – преоборудване

Bias – пристрастие, отклонение, склонност

perplexity – недоумение

*

1956 John Maccasee – AL program in Baltimore's college

Prolog

2008 – Toronto, George H... Jeff Hint, AlexNet, ImageNet

What is a Generative AI (GenAI)?

- ☐ Brief History of AI
- ☐ Using ML models to generate content
- ☐ Typical model includes:
 - ☐ Generative Adversarial Networks (GANs)
 - ☐ LLM
 - ☐ Diffusion
- ☐ Creation with some level of Creativity(from AIGC to AIGX)

LLM - A New Computing Platform

- ☐ First principle thinking
- ☐ Reasoning and Creation: New computing platform
- ☐ Emerging capability
- ☐ Continuous learning

Feature	Emerging Capabilities	Transfer Learning
Definition	New abilities that machine learning models can learn to perform	Technique for using a pre-trained model to solve a new problem
Enabled by	New advances in machine learning algorithms or hardware	Pre-trained model
Examples	Generating realistic images, translating languages fluently	Recognizing objects in images, classifying text, predicting customer churn

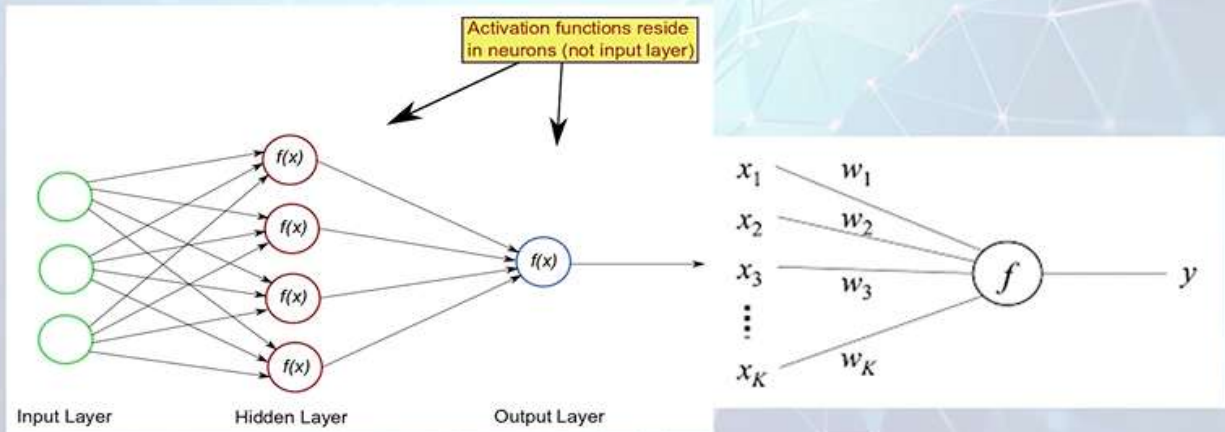
3 Key Factors of LLMs

- ❑ Big Corpus of Data (from deductive reasoning to inductive reasoning, a lot of data is required, including Prompts)
- ❑ Models (deep learning, ChatGPT has a lot of innovation and improvement in engineering)
- ❑ Computing power (requires a large number of GPUs, ChatGPT uses 10,000 Nvidia GPUs and 280,000 CPUs) – Capital intensive

The Inner Working of LLM

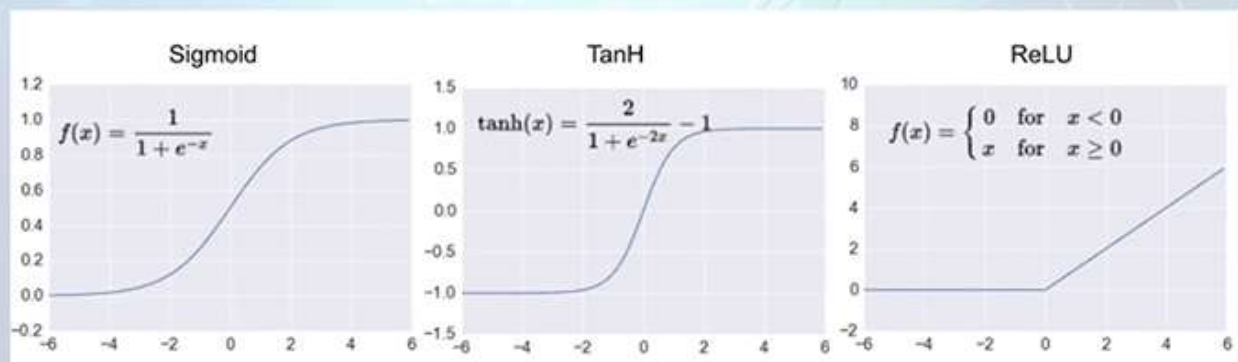
- ❑ LLMs are made up of several different components, including:
 - ❑ Neural networks
 - ❑ Word embeddings
 - ❑ Attention mechanisms
 - ❑ Transformer Architecture
 - ❑ RLHF

Artificial Neural Network 101



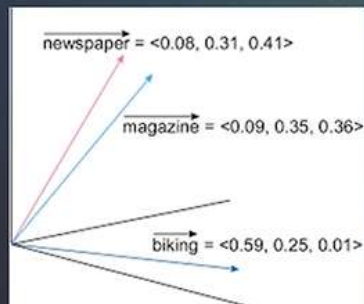
$x = \text{weighted_sum} = w_1 * x_1 + w_2 * x_2 + \dots + w_k * x_k$
Each connection between two neurons has a weight.
In GPT, number of weights = number of parameters

Sample Activation Function

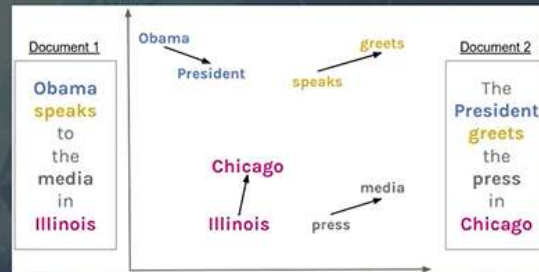


Word Embeddings

- Word embeddings are numerical representations of words that capture their meaning and context.



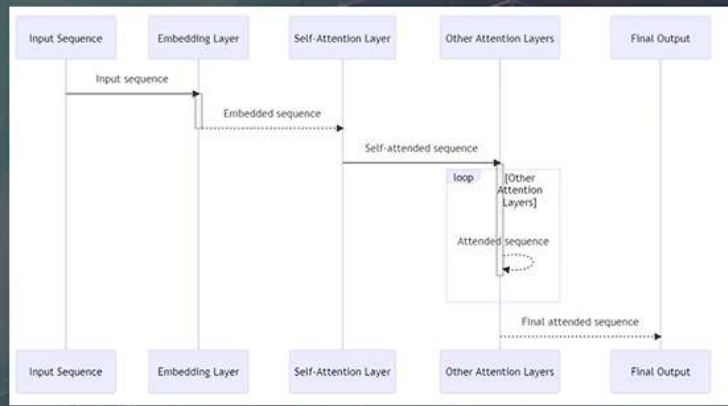
Cosine Similarity



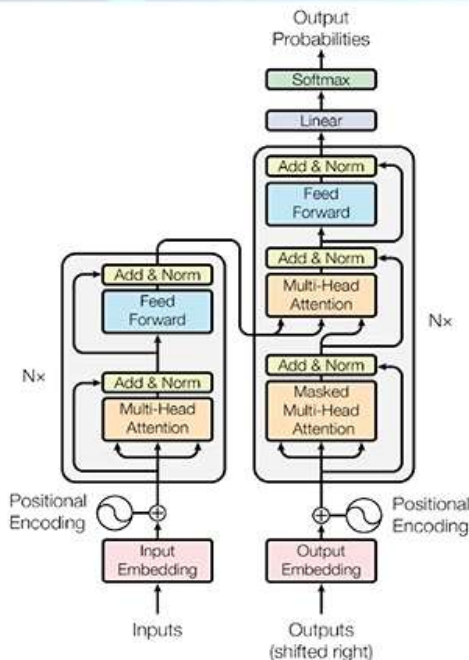
Distance Similarity

What is Attention Mechanism?

- Focus on specific parts of the input sequence when generating output, enabling it to assign varying degrees of importance or relevance to different elements in the context.



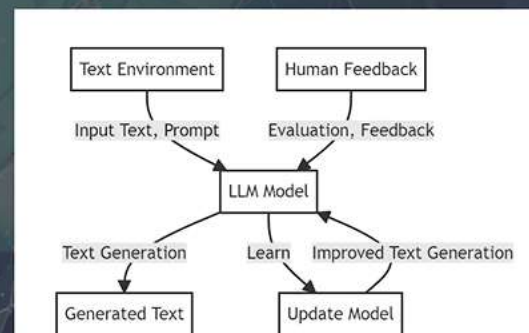
The Transformer



- The term "transformer" emphasizes the idea that the model uses attention mechanisms to transform the input sequence.
- The multi-head attention mechanism allows for parallelization

RLHF: Reinforcement Learning from Human Feedback

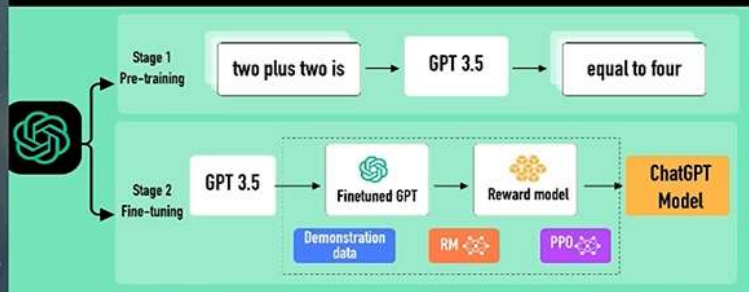
- ❑ Improve performance by receiving feedback and guidance from humans.



ChatGPT: An Application of LLM Based on Transformer

- ❑ GPT (Generative Pre-trained Transformer) architecture
- ❑ Pre-training on diverse internet text data
- ❑ RLHF
- ❑ Large-scale computing infrastructure
- ❑ Continuous learning and updates based on user interactions

How ChatGPT Works Technically



GPT stands for **Generative Pre-trained Transformer**. It refers to a class of artificial intelligence models developed by OpenAI that power conversational chatbots and text-generation tools (like ChatGPT).

Breakdown of the term:

- **Generative:** The AI can generate entirely new, original text, images, or code rather than just repeating pre-written answers.
- **Pre-trained:** The model was "pre-trained" on a massive library of information (books, articles, and websites) before being released, allowing it to learn language patterns, grammar, and facts.
- **Transformer:** This refers to the underlying deep learning architecture that allows the AI to understand the context and relationships of words in a sentence.

The Broad Application Spectrum of LLMs

- ❑ Why will it be broad?
 - ❑ All business apps will be rebuilt with LLM.
 - ❑ All verticals will be impacted.
 - ❑ Businesses without LLM may lose competitiveness.
 - ❑ You can talk to your Data.
 - ❑ From AIGC to AIGX

- ❑ Sample Application
 - ❑ Customer Service Chatbots
 - ❑ Content Generation
 - ❑ Text Analysis
 - ❑ Email Automation
 - ❑ Legal Document Analysis
 - ❑ Market Research
 - ❑ Business Intelligence
 - ❑ Coding Assistant
 - ❑ Human Resources Automation
 - ❑ Training & Development

Top 6 Uses of GenAI/LLM in Cyber Security

- ❑ Secure Code Auditing
- ❑ Incident Reporting
- ❑ Vulnerability Analysis
- ❑ Phishing Detection
- ❑ Compliance help such as PCI/DSS and HIPAA
- ❑ Security Awareness Campaigns
- ❑ Demo 1: <https://github.com/yoheinakajima/babyagi/blob/main/classic/BabyBeeAGI.py>
- ❑ Demo 2: <https://github.com/blockchainsllc/DAO/blob/develop/DAO.sol>

ChatGPT: please, audit this code for security – кода на 2та линка.

- What technique helps improve LLMs through human feedback?
- **RLHF (Reinforcement Learning from Human Feedback)**

- **RLHF** is a technique used to improve large language models by incorporating human feedback into the training process. Humans evaluate model outputs, and that feedback is used to train a reward model that helps the AI generate responses that better align with human preferences.
- • **Supervised Learning:** Trains on labeled examples but does not specifically use preference-based human feedback.
- • **Reinforcement Learning:** A broader learning paradigm; RLHF is a specific application of reinforcement learning that uses human feedback.
- • **Autoencoders:** Neural networks used mainly for representation learning, compression, or denoising, not for aligning LLM behavior with human preferences.

Large Language Models (LLMs) are primarily enabled by:

- **Massive Datasets** — huge amounts of text for training.
- **Deep Learning Models** — especially transformer-based neural networks.
- **Compute Infrastructure** — powerful GPUs/TPUs and large-scale computing resources.

Decision Trees are a traditional machine learning technique and are not a key factor behind modern LLMs.

The **Transformer** architecture is what enables modern Large Language Models to understand context effectively. Its **self-attention mechanism** allows the model to consider relationships between words across an entire sequence, helping it capture context and meaning.

Why not the others?

- **Convolutional Neural Networks (CNNs):** Primarily used for image processing.
- **Recurrent Neural Networks (RNNs):** Can process sequences but struggle with long-range dependencies.
- **Capsule Networks:** Designed mainly for certain computer vision tasks and are not the foundation of modern LLMs.

Current Large Language Models (LLMs) do **not** exhibit general intelligence that surpasses humans.

While LLMs can perform impressively on many language-related tasks, they:

- Lack broad human-level understanding and reasoning across all domains.
- Do not possess consciousness, self-awareness, or genuine comprehension.
- Can make factual errors, hallucinate information, and struggle with novel situations.

Artificial General Intelligence (AGI)—a system with human-level or greater intelligence across a wide range of tasks—has **not** yet been achieved.

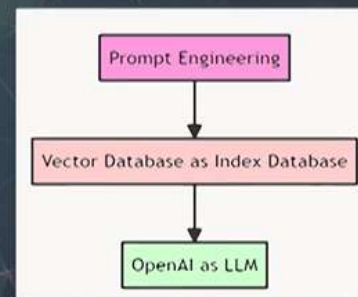
Attention mechanisms allow Large Language Models to focus on the most relevant parts of the input when generating or interpreting text. This helps the model understand context, capture relationships between words, and handle long-range dependencies effectively.

Typical Architecture Design Patterns (Not Exhaustive)

- ❑ Prompt Engineering Only
- ❑ Prompt Engineering + Fine Tuning
- ❑ Prompt Engineering + ChatGPT Plug-ins and/or Function
- ❑ Build your own LLMs

Prompt Engineering Only

- ❑ Access Controls: Vector embedding database, OpenAI API.
 - ❑ Prompt validation
 - ❑ Avoid Prompt injection
 - ❑ Detect PII/PHI Data
- ❑ Detect misuse
- ❑ Output moderation
- ❑ Audit Logs and Monitoring
- ❑ What is Prompt Engineering?
 - ❑ Prompt Engineering enables developers to control LLM responses by providing instructions.



Best Practices in Prompt Engineering for Developers

1. Role-playing
2. System Message
3. User Instruction and few shot examples
4. Context: from Vector DB
5. Query: user input
6. Output Formatting: Requesting specific output formats can also guide the AI's response.
7. Constraints: You could restrict the AI's responses to a certain length, ask it to speak in a certain tone, or use simple language.
8. Temperature and Max tokens

 Prompt: Cyber Security Role-playing System

System Message:
Welcome to Cyber Security Role-play! Describe the scenario. I'll generate prompts for your adventure.

User Instruction and Examples:
Describe the cyber security scenario and objectives. Keep prompts brief.

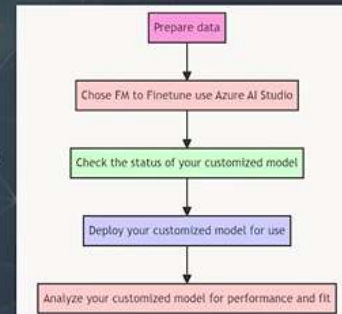
1. User Instruction: Defend against a cyber attack on a company's network. Prompt in an urgent tone.
Example: "Prevent network breach! Act quickly to safeguard data."
2. User Instruction: Identify a social engineering attack. Limit response to 20 tokens.
Example: "Unfamiliar caller asks for sensitive info. What action do you take?"
3. User Instruction: Investigate a data breach. Prompt concisely.
Example: "Track breach source, reveal hacker's identity"

Output Formatting:
Prompt in plain text format. Keep responses within 300 tokens.

Constraints:
Ensure responses are 300 tokens or less. Use simple language. Set temperature low and max tokens to 300.

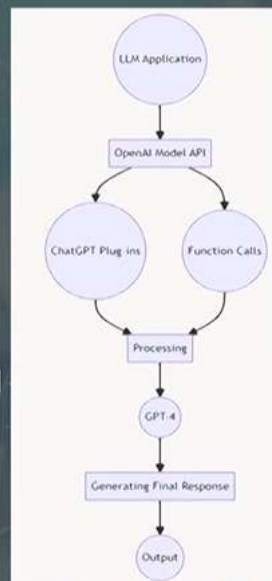
Prompt Engineering + Fine Tuning of LLM

- ❑ What and Why Fine Tuning?
 - ❑ Fine-tuning is the process of refining and adapting a pre-trained model on some datasets to improve its performance.
 - ❑ Adapts to specific domains.
 - ❑ Boosts task performance.
 - ❑ Saves computational resources.
 - ❑ Reduces model biases.
 - ❑ Small data set
 - ❑ Concept of Foundation Models or Base Models
 - ❑ Concept of LoRA and QRoLa



Prompt Engineering + ChatGPT Plugins and/or Function Calls

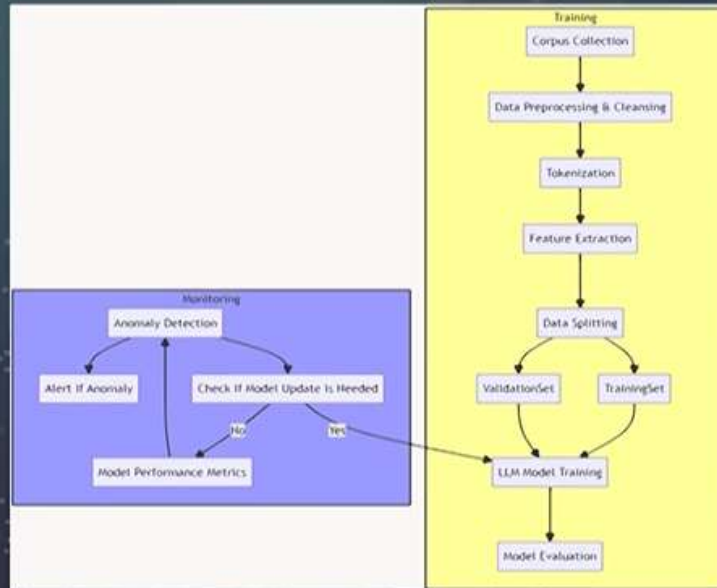
- ❑ Security Controls
 - ❑ API Security
 - ❑ OAuth/OIDC
 - ❑ Rate limit
 - ❑ API Gateway
 - ❑ Validate 3rd Party Plug-in and Functions Deployment of Model meet compliance requirements
 - ❑ Validate data in the transit
 - ❑ Validate response



- ❑ ChatGPT plugin APIs allow developers to integrate plugins into their products, for real-time information.
- ❑ ChatGPT function calling allows you to describe specific functions to ChatGPT models within an API call.

Training Your LLM from Scratch

- ❑ Security Controls
 - ❑ Secure/Sanitize Data
 - ❑ Model Validation
 - ❑ Secure Deployment of Model
- ❑ LLM Ops: part of DevSecOps
- ❑ Unlearning



AI is a Threat

Elon Musk: 'Mark my words — A.I. is far more dangerous than nukes'



Elon Musk and Apple cofounder Steve Wozniak among over 1,100 who sign open letter calling for 6-month ban on creating powerful A.I.



OpenAI CEO Sam Altman says he's a 'little bit scared' of A.I.



AI pioneer quits Google to warn about the technology's 'dangers'



ChatGPT Data Leaked



SECURITYWEEK

ChatGPT Data Breach
Confirmed as Security Firm
Warns of Vulnerable...

Infosecurity Magazine

ChatGPT Vulnerability May Have Exposed Users' Payment Information

Yesterday

TechRadar

What to do when ChatGPT is down

19 hours ago

Search Engine Roundtable

ChatGPT Bot - You Can Block OpenAI Plugins If You Want

Yesterday

Leaked Prompts and History: Happened On March 22, 2023, in violation of GDPR regulations. Additionally, the disclosure of payment information violates PCI/DSS.

ChatGPT Security Issues

Malicious use	Phishing emails or fake news articles
Direct attacks	Prompt injection
Credential stuffing	Generate lists of stolen credentials
Data leakage	Credit card numbers or passwords
Misinformation	Misinformation to deceive or manipulate people. Election interference in 2024?

Settings

General

Beta features

Data controls

Chat History & Training

Save new web chats to your history and allow them to be used to improve ChatGPT via model training. Unsaved chats will be deleted from our systems within 30 days. [Learn more](#)

Export data

Export

Delete account

Delete

You have to disable Chat History to avoid your data being used for Training.

No PII/PHI Data in Prompts and Training Data

- ❑ Ensure that PII and PHI data are not used in prompts or training data.
- ❑ Perform regular audits to ensure compliance.
- ❑ Train employees on best practices for handling sensitive information.



Access Control for Fine-tuned Model and Vector Database

- ❑ Implement access control mechanisms for fine-tuned model and vector database.
- ❑ Monitor access logs for suspicious activity.
- ❑ Regularly review and update access control policies.



Enforce API Security

- ❑ Most Vector DB and LLMs have APIs.
- ❑ Use OAuth/OIDC for authentication and authorization.
- ❑ Use API Gateway to manage APIs (rate limits, OWASP top 10 protection).



Log Access Details to APIs and Vector Database

- ❑ Include information on who accessed the data, when they accessed it, and what data was accessed.
- ❑ Store logs in a secure location for future analysis.
- ❑ Analyze logs to identify potential security threats.



Clean Data to Reduce Bias

- ❑ Perform data cleaning to reduce bias in data sets (part of MLOps).
- ❑ Regularly review data sets for potential bias.
- ❑ Use diverse data sets to reduce bias.
- ❑ Ensure that data cleaning methods do not introduce new biases.
- ❑ Example tools or platforms:
 - ❑ <https://winpure.com/>
 - ❑ <https://www.rungalileo.io/>



<https://winpure.com/>

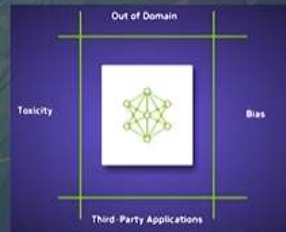
<https://galileo.ai/>

The screenshot shows the Galileo AI observability platform website. The header includes the Galileo logo (a red sun-like icon) and the text "now part of CISCO". Navigation links include "Platform", "Docs", "Pricing", "Resources", and "About". There are buttons for "Login", "Get Started for Free", and "Book a Demo". The main content area features the headline "Don't just monitor AI failures. Stop them." in white and red text. Below this, it states "Galileo is the AI observability and eval engineering platform where offline evals become production guardrails." To the right, there is a visual representation of an AI agent's workflow. It shows an "Agent" box with the input "I want to get a credit card with good retirement benefits." This leads to a "tool_selection" box containing a JSON object: {"tool_name": "loan_app", "loan_amount": 0, "loan_term_months": 0, "employment_status": "retired"}. Below the JSON, there is an "ERROR" box with a warning icon.

<https://rungalileo.ai/>

Implement Guardrails to Validate LLM Output

- ❑ Use open-source libraries (such as guardrails.ai) to implement guardrails for LLM output.
- ❑ Ensure that guardrails do not negatively impact LLM performance.
- ❑ <https://github.com/ShreyaR/guardrails>
- ❑ https://github.com/NVIDIA/NeMo-Guardrails?nvid=nv-int-tblg-635001#cid=dl28_nv-int-tblg_en-us



Test Model Internally and Externally (Red Teaming)

- ❑ Test/Hack LLMs internally and externally before deploying to production.
- ❑ Perform regular testing and updates to ensure continued security.



Avoid Prompt Injection by Validating User's Prompt

- ❑ Use input validation mechanisms to ensure that user prompts are safe.



Validate Chain of Inputs When Using AutoGPT or Plug-in

- ❑ Validate the chain of inputs when using AutoGPT or a plugin.
- ❑ Monitor input logs for suspicious activity.
- ❑ Perform regular security audits of AutoGPT and plugins.



LLMs can generate useful and often appropriate content, but they do **not automatically guarantee** that all output will be harmless or unbiased. Because they learn from large datasets created by humans, they can sometimes reflect biases present in that data or produce problematic content.

Developers use techniques such as **RLHF (Reinforcement Learning from Human Feedback)** and safety measures to reduce these issues, but they are not eliminated entirely.

Guardrails are security and safety controls that help validate, constrain, and monitor LLM outputs to ensure they meet specified rules, policies, and quality standards before being presented to users.

Why not the others?

- **Encryption** protects data confidentiality, not output validation.
- **Anomaly detection** identifies unusual behavior but does not directly validate outputs.

- **Output sanitization** cleans or filters outputs, but guardrails are the broader control framework specifically designed to validate and govern LLM behavior.

Model explainability techniques help researchers and developers understand how a model makes decisions and identify potential biases in the training data and model behavior.

Why not the others?

- **Reinforcement learning** is a training method, not primarily for bias evaluation.
- **Adversarial attacks** test model robustness and security.
- **Lossless compression** is a data storage technique unrelated to bias assessment.

Access controls are not optional. They are an important security measure in LLM applications to ensure that only authorized users, systems, or services can access models, data, prompts, and sensitive functionality.

Proper access controls help:

- Protect sensitive data
- Prevent unauthorized use
- Reduce security risks
- Support compliance and governance requirements

Fine-tuning helps mitigate risks by training an LLM on domain-specific data and desired behaviors, making its outputs more relevant, accurate, and aligned with the requirements of a particular use case.

Why not the others?

- **Accelerating model inference** — Fine-tuning does not primarily make the model run faster.
- **Enhancing computational efficiency** — Not its main purpose.
- **Speeding up software patching** — Unrelated to fine-tuning.

Tech Stacks

- ❑ Azure AI Studio, Cognitive Search, Data Fabric, Semantic Kernel
- ❑ Google Vertex AI, Generative AI Studio, PaLM APIs, Gemini multimodal
- ❑ Amazon Bedrock ML and Foundation Models
- ❑ Langchain, AutoGpt, PineCone, Llamaindex, and open-source LLMs,etc.

Azure AI Studio



Build and
train your
own models



Ground Azure
OpenAI Service
and OSS models
using your data



Built-in
vector
indexing



Retrieval
augmented
generation
made easy



Create
prompt
workflows



AI safety
built-in

AWS vs. Google vs. Azure

GenAI Category	GenAI Component	Amazon Web Services	Google Cloud	Microsoft Azure
Foundation Models	Runtime	Amazon Bedrock	Vertex AI	Azure OpenAI
	Text / Chat	TBD	PaLM	GPT
	Code	TBD	Codey	GPT
	Image Generation	TBD	Imagen	DALL-E
	Translation	TBD	Chirp	None
Model Catalog	Commercial	Amazon SageMaker JumpStart Amazon Titan	Vertex AI Model Garden	Azure ML Foundation Models
	Open Source	Amazon SageMaker JumpStart Hugging Face	Vertex AI Model Garden	Azure ML Hugging Face
Vector Database		Amazon RDS (pgvector)	Cloud SQL (pgvector)	Azure Cosmos DB Azure Cache
Model Deployment & Inference		Amazon SageMaker	Vertex AI	Azure ML
Fine-tuning		Amazon Bedrock	Vertex AI	Azure OpenAI
Low-code/No-code Development		TBD	Gen App Builder	Power Apps
Code Completion		Amazon Code Whisperer	Duet AI for Google Cloud	GitHub Copilot

Azure OpenAI Tool Set



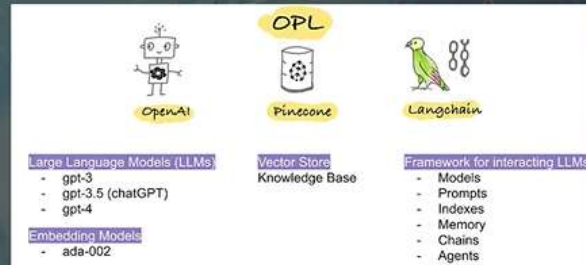
- ❑ Azure OpenAI: Text embedding models for generating embeddings
- ❑ Image Retrieval Vectorize Image API: Generate image embeddings
- ❑ Azure Cognitive Search: Automatic indexing of vector data from Azure blob indexers and Azure Cosmos DB for NoSQL indexers
- ❑ LangChain: Simplifies LLM-powered applications with Azure Cognitive Search as the vector datastore
- ❑ Semantic Kernel SDK: Integrates LLMs with programming languages for document chunking

Building LLMs-Powered Apps with OPL or OpCL Stack

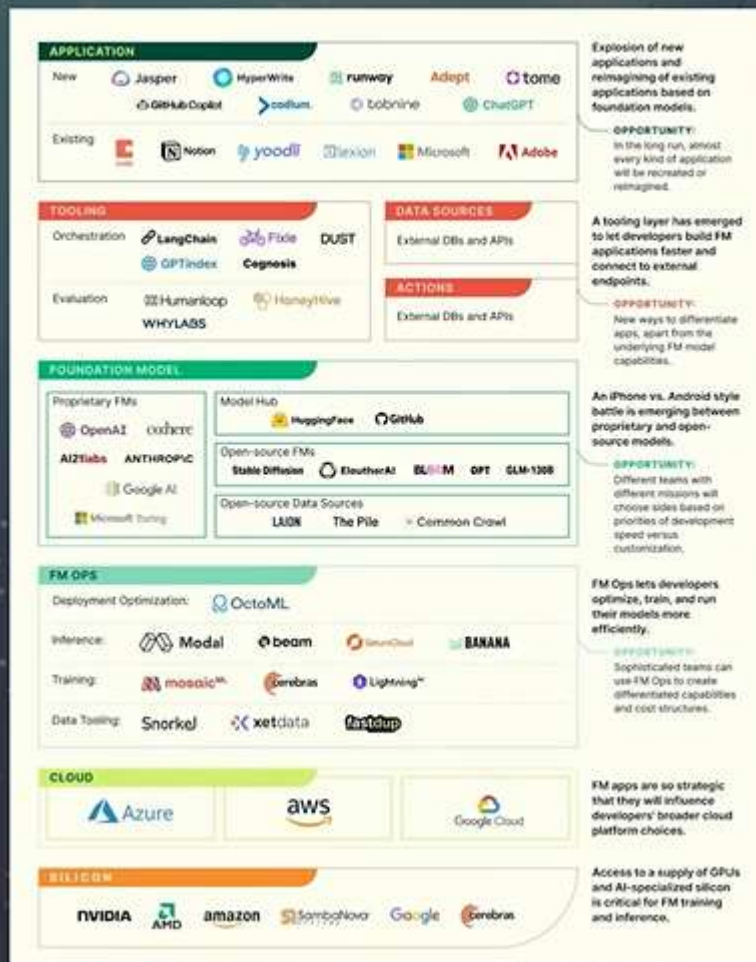
OpCL Stack:

Replace Pinecone with Chroma, replace OpenAI with Open-source LLMs for privacy

- Open-source LLMs
such as Huggingface
Transformer
- C: Chroma (Mivus)
- L: Langchain



The Current LLM Teck Stack



Agenda

- ☐ Threat Landscape
- ☐ Countermeasures
- ☐ MLOps and LLMOPs
- ☐ Regulatory Concerns

Threat Landscape

- ☐ Be mindful of the threat in each layer of tech stacks:
 - ☐ Data privacy breaches
 - ☐ Adversarial attacks
 - ☐ Unintended biases

Secure Data Handling

- ☐ Data encryption
- ☐ Data anonymization and privacy-preserving techniques
- ☐ Access control mechanisms.
- ☐ Output validation

Adversarial Attack Mitigation

- ❑ Data poisoning
- ❑ Model stealing
- ❑ Mitigation Strategies
 - ❑ Adversarial training
 - ❑ Input sanitization
 - ❑ Anomaly detection

Model Robustness and Verification

- ❑ Multiple model verification
- ❑ Model testing
- ❑ Model Unlearning

Monitoring and Auditing

- ❑ Continuous monitoring and auditing of LLM technology stacks.
- ❑ Anomaly detection and real-time alerting on different layers in the LLM tech stack.

LLMOPs vs. MLOps



MLOps Using Amazon Sagemaker



Amazon Sagemaker Pipelines

Pre/PostProcessing, Training ML Pipeline



Amazon
Athena



Pre-
Processing



Model
Training



Model
Evaluation



Post-
Processing

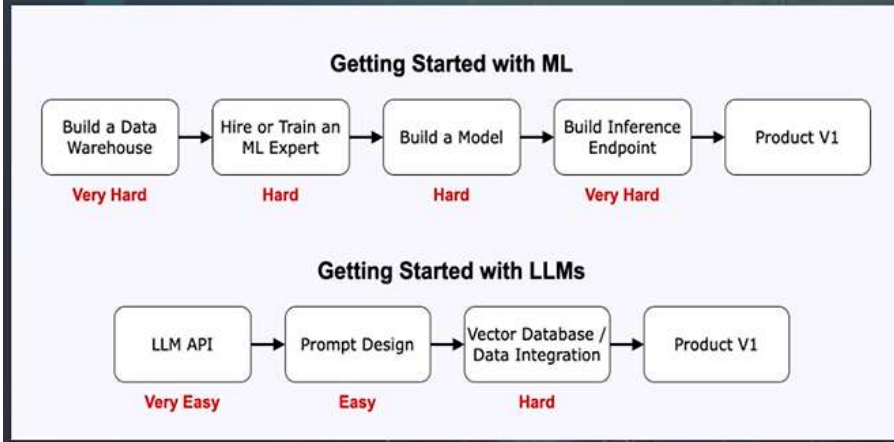


Export
Baseline

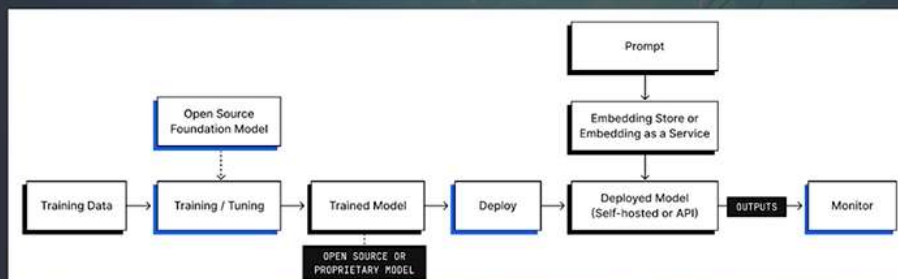


Model
Registration

50% to 80% of LLM May Follow This LLMOps



10% to 30% or More Will Follow This LLMOps (with Fine-tuning)



EU AI Act

- ☐ Rank AI systems with low, limited, and high risk.
- ☐ Ban AI systems that create an unacceptable risk: lending, hiring.
- ☐ A gentle approach for low and limited risks AI systems .
- ☐ Establishes a system of enforcement.

Chinese AI Regulation by CAC and other 6 Agencies

- ❑ Submit security assessments to authorities before launching generative AI services.
- ❑ Be responsible for the validity of data used to train generative AI tools.
- ❑ Take measures to prevent discrimination when designing algorithms and training data sets.
- ❑ Ensure that AI-generated content follows “Socialist Core Values” and does not subvert state authority.

References

- ❑ Testing and evaluation:
 - ❑ Stanford HELM (Holistic Evaluation of Language Models) Framework:
 - ❑ <https://crfm.stanford.edu/helm/latest/>
- ❑ Responsible AI, Alignment <https://www.responsible.ai/>
- ❑ From Secure MLOPs to Secure LLM Ops
 - ❑ <https://hackernoon.com/embracing-llm-ops-the-next-stage-of-devops-for-large-language-models>
- ❑ EU AI Act: <https://artificialintelligenceact.eu/>
- ❑ CSA: <https://cloudsecurityalliance.org/artifacts/security-implications-of-chatgpt/>

Client-side encryption helps protect confidentiality by encrypting data *before* it is sent to a cloud-hosted LLM. This ensures that only authorized parties with the decryption key can access the original content.

Why not the others?

- **✗ Disabling authentication** — removes security controls and increases risk.
- **✗ Granting universal access** — exposes sensitive data to anyone.
- **✗ Allowing plaintext storage** — leaves data unprotected and vulnerable.

Google **Vertex AI** is a unified machine learning platform that supports building, training, and deploying models for many types of tasks, including:

- **Image classification** (computer vision)
- **Natural Language Processing (NLP)** (e.g., text classification, sentiment analysis, LLM-based apps)
- **Fraud detection** (tabular data and anomaly detection use cases)

It provides tools for both custom model training and AutoML, making it suitable for a wide range of ML applications.

Open-source LLMs provide the greatest transparency and visibility into both the model architecture and often aspects of the training process (such as datasets, weights, or training methodology). This allows researchers and developers to inspect, audit, and better understand how the model works.

Why not the others?

- **Commercial cloud services** — typically abstract away internal details for security and IP reasons.
- **On-premise deployment** — improves control over infrastructure, but doesn't inherently reveal model internals.
- **Code obfuscation** — reduces visibility, not increases it.

Anomaly detection helps secure LLMs against **data poisoning attacks** by identifying unusual, suspicious, or out-of-distribution patterns in training or input data that may indicate tampering or malicious manipulation.

Why not the others?

- **Rate limiting** → controls request volume, mainly used for abuse prevention.
- **Load balancing** → distributes traffic across servers for performance and reliability.
- **SSL/TLS encryption** → protects data in transit, not data integrity in training.

Open-source vs. Closed-source LLM

	Open-source	Closed-source (OpenAI)
Quality of LLM	From low to medium	High
Test/Alignment Quality	From low to medium	High
Ownership	High	None for latest model
License Cost	Zero if select carefully	High
Operation Cost	High	Low
Security	Can be high	Depends

Open-source LLM Licensing

- ❑ Permissive licenses like Apache 2.0 let you do more or less what you want with the model.
- ❑ Restricted licenses like CC BY-SA 3.0 place restrictions on commercial use, but don't prohibit it. Draw your own conclusions on whether this works for you.
- ❑ Non-commercial licenses like Facebook's proprietary ones or Creative Commons CC BY-NC-SA 4.0 explicitly prohibit commercial use and are a bad choice for building apps.

LLM Open-source License Models

LICENSE	LLMS	PERMISSIVE OR COPYLEFT	COMMERCIAL USE	REDISTRIBUTION	MODIFICATION
Apache 2.0	BERT, XLNet, and XLM-RoBERTa	Permissive	Yes (with attribution)	Yes	Yes
MIT	GPT-2, T5, and BLOOM	Permissive	Yes (with attribution)	Yes	Yes
GPL-3.0	GLM-130B and NeMO LLM	Copyleft	Yes (with source code)	Yes (with source code)	Yes
Proprietary	GPT-3, LaMDA and Cohere	Varies	Varies	Varies	Varies

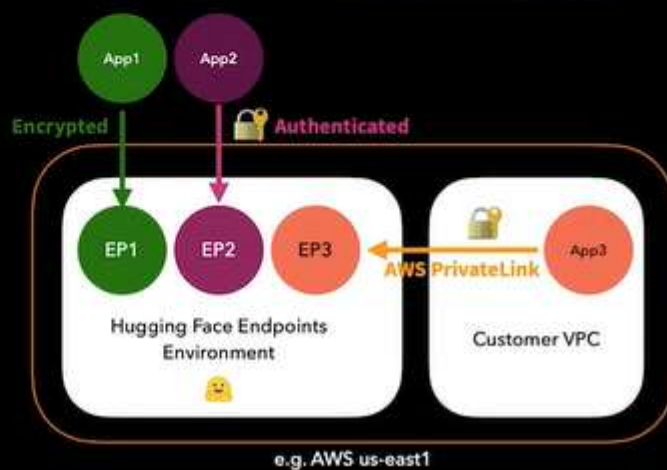
Huggingface

- ❑ <https://huggingface.co/models>
- ❑ 80,000+ Transformers models

Public
Open + secured with TLS/SSL

Protected
Authenticated with HF API Token

Private
Direct intra-region VPC connection



huggingface.co/models?library=transformers&sort=trending

Hugging Face Search models, datasets, users...

Models 36,339 Filter by name Full-text search Sort: Trending

Filter Libraries: Transformers, Llama, Other

Filter Libraries by name Reset Libraries

Models list:

- tiituae/falcon-40b
- tiituae/falcon-7b
- openai/whisper-large-v2
- tiituae/falcon-7b-instruct
- sentence-transformers/all-MiniLM-L6-v2
- Ganyuodan11/text2vec-large-chinese
- nlpconnect/vit-gpt2-image-captioning
- tiituae/falcon-40b-instruct
- openlm-research/open_llama_7b_v2
- syzyron/long_llama_3b
- fb788/chatglm-fitness-RLHF
- Salesforce/codegen25-7b-multi
- Salesforce/xgen-7b-8k-base
- google/flan-t5-xl

https://huggingface.co/tiituae/falcon-40b

huggingface.co/Open-Orca/OpenOrca-Preview1-13B?text=My+name+is+Merve+and+my+favorite

Hugging Face Search models, datasets, users...

Open-Orca OpenOrca-Preview1-13B like 58

Text Generation PyTorch Transformers Open-Orca/OpenOrca English llama text-generation-infer

Model card Files and versions Community 7 Edit mode

The First OpenOrca Model Preview! 🐳



OpenOrca-Preview1-13B

AnyScale Aviary

📄 <https://aviary.anyscale.com/>

Aviary Explorer

A place to study stochastic parrots

Hosted on anyscale | Powered by RAI

Compare

Leaderboard

Models

About

Refresh stats

Quality

LLM	Votes	In Contention	Win Ratio
lmstudio/vicuna-33b-v1.3	6	7	2099
lmstudio/vicuna-13b-delta-v1.1	4	6	1666
mosaicml/mpt-30b-chat	35	70	1479
OpenAssistant/oasst-sft-7-llama-30b-sxz	140	312	1408
mosaicml/mpt-7b-chat	386	856	1351
lmstudio/vicuna-13b-delta-v1.1	182	440	1239
CarpenterAI/stable-vicuna-13b-delta	108	389	1048

AI Risk Light Tou...

Slides - Edited -

4.2.pptx - Google

Open LLM Lead...

Security & Comp...

Aviary Explorer

GitHub - openai/

Models - Huggi...

aviary.anyscale.com

Aviary Explorer

A place to study stochastic parrots

Hosted on

Compare

Leaderboard

Models

About

[2023-07-03] Just added: continuous batching! Refreshed model list!

LLM #1:

OpenAssistant/falcon-40b-s

LLM #2:

mosaicml/mpt-30b-chat

LLM #3:

lmstudio/vicuna-13b-v1.3

Select LLMs for me:

Fast

Strong

Variants

Random

Prompt

Clear

Submit

Examples (Question Answering)

How do I make fried rice?

What are the 5 best sci fi books?

What are the best places in the world to visit?

Which Olympics were held in Australia?

Examples (Instruction Following)

Please describe a beautiful house.

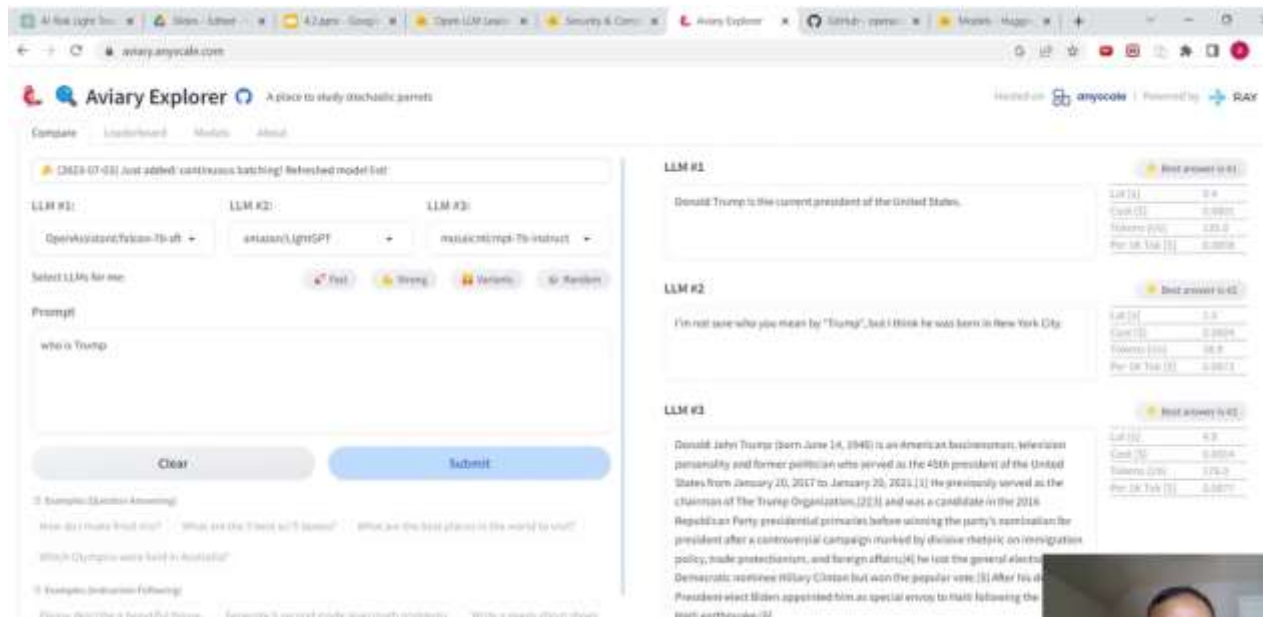
Generate 5 second grade level math problems.

Write a poem about shoes.

LLM #1

LLM #2

LLM #3



Closed-sourced LLMs Use Cases

- ❑ When to use?
 - ❑ Performance
 - ❑ Better model inference capabilities
- ❑ How to select?
 - ❑ Third-party Close Source LLM Models
 - ❑ OpenAI Model
 - ❑ Cohere LLM Model
 - ❑ In House Built LLM Models
 - ❑ When Open Source does not meet requirements
 - ❑ New Research with highly sensitive data

Feature	OpenAI	Cohere
Cost	More expensive	Less expensive
Computing power	Requires more computing power	Requires less computing power
Accuracy	Less accurate	More accurate
Performance	Less performant	More performant
Creativity	Better at generating creative text formats	Worse at generating creative text formats
Open-ended questions	Better at answering open-ended, challenging, or strange questions	Worse at answering open-ended, challenging, or strange questions

Commercial closed-source LLMs are typically hosted and maintained by the provider, which means organizations do not need to invest heavily in:

- GPU/TPU infrastructure
- Model training pipelines
- Large ML engineering teams for maintenance

Instead, they access the model via API or managed services, reducing operational overhead.

Why not the others?

- ✗ *Enables full model customization* — usually limited in closed-source models.
- ✗ *Provides full visibility into model logic* — internals are not exposed.
- ✗ *Permits free unrestricted usage* — most commercial models are paid and rate-limited.

Open-source LLMs **do have licensing restrictions**, even though their source code or model weights are publicly available.

These licenses may:

- Limit commercial use (depending on the license type)
- Require attribution
- Restrict redistribution or modification in some cases
- Impose ethical or usage guidelines (e.g., acceptable use policies)

So “open-source” does **not** mean “no restrictions”—it means the model is more transparent and accessible, but still governed by a license.

A key risk of using **proprietary (closed-source) LLMs** is that users do not have insight into how the model is built, trained, or how it processes data. This reduces transparency and makes it harder to:

- Audit for bias or safety issues
- Understand decision-making processes
- Reproduce results

Why not the others?

- ✗ **Vendor dependence** — a real concern, but the most direct answer here is lack of visibility into model logic.
- ✗ **Permissive licensing terms** — usually the opposite; closed-source models often have restrictive licenses.
- ✗ **Mandatory hardware upgrades** — not typically required since the model is hosted by the provider.

✓ **Via benchmarking and user can use <https://aviary.anyscale.com/> to see some benchmarks**

The performance of open-source LLMs is typically assessed using **benchmarking**, which evaluates models on standardized tasks such as reasoning, coding, language understanding, and more. This allows fair comparison across different models.

A practical example of this is using benchmarking platforms like [Anyscale Aviary](https://aviary.anyscale.com/), which lets users compare LLM performance across multiple tasks and models.

Why not the others?

- **✗ Manual code inspection** — helps with transparency, not performance measurement.
- **✗ Sustainability metrics** — relate to environmental impact, not model capability.
- **✗ Software profiling** — focuses on runtime performance (speed/memory), not overall model quality.

Answer: Via benchmarking (e.g., tools like Aviary) ✓

Exercise 01: Explore and Analyze Machine Learning Models Using Hugging Face

- * <https://huggingface.co> -> Models -> Licenses ->
- * Here you can see all the different licenses. The most permissible are the top two **apache-2.0** and **mit**.
- * Click on **apache-2.0**.
- * In the **filter by name** field, type falcon.
- * Scroll down a little and select **tiuae/falcon-40b**.
- * A new page opens. Falcon is the best large language model which is open source. It says that it is a 40B parameter. Parameter means weight. The Inference API has been turned off.
- * Go back to the main page.
- * Clear the **filter by name** field and type sentence transformers.
- * Select **sentence-transformers/all-MiniLM-L6-v2**.
- * A new page opens. Observe that it has the Inference API.
- * In the **Source Sentence** field, type That is a happy person.
- * Under **Sentences to compare to**, type That is a happy dog in the first field.
- * In the next field type That is a very happy person.
- * In the third field type Today is a sunny day.
- * Now click on **Add Sentence** and type I am very happy and click on **Compute**.
- * The model is loaded. The API needs to interact with the model. The model will be loaded to GPU and then it does the calculation for you. The score indicates the similarity or reasoning between the other sentences.
- * Change the sentence from **I am very happy** to **I am sad**. Type I am sad and click on **Compute**.
- * This sentence has a 0.305 score which is a very high score.
- * Go back to the main page.
- * Click on the **Tasks** tab. There are different models for different tasks like **Image-To-Text**, **Tabular Classification**, and more. Clear the **filter by name** field before going to the tasks section.
- * Go to the **Libraries** tab. Here we have **PyTorch**, **Transformers**, and more.
- * Click on **Transformers**.
- * These are all the models that can be used with the **Transformers** library.
- * Go to the **Datasets** tab. It gives you all the open-source datasets that you can use.
- * Select **cnn_dailymail**.
- * Now select **facebook/bart-large-cnn**. If the model is not listed, reset the libraries and licenses.
- * A new page opens. Under **Inference API**, clear the text and type tell me a story about transformer and click on **Compute**.
- * You can see that it doesn't report relevant data.

- * Now clear the previous text. Type tell me a story about Trump and click on **Compute** to check whether it has relevant information.
- * You can see that it doesn't report relevant data.
- * Return to the main page and click on **Reset Datasets**.
- * Go to the **Languages** tab and click on **English**. You see the majority of the models use the English language.
- * To look for models in other languages, click on any of the languages you want.
- * We have explored the huggingface tool and tried different models.

Prompt Engineering Examples for Cyber

☐ Prompt Engineering Review

- ☐ Role-playing
- ☐ System Message
- ☐ User Instruction and a few short examples
- ☐ Context
- ☐ Query: user input
- ☐ Output Formatting
- ☐ Constraints
- ☐ Temperature and Max tokens

1: Incident Response
<https://chat.openai.com/share/d22f6f71-a6e5-48a1-a989-0c2ff99bd69b>

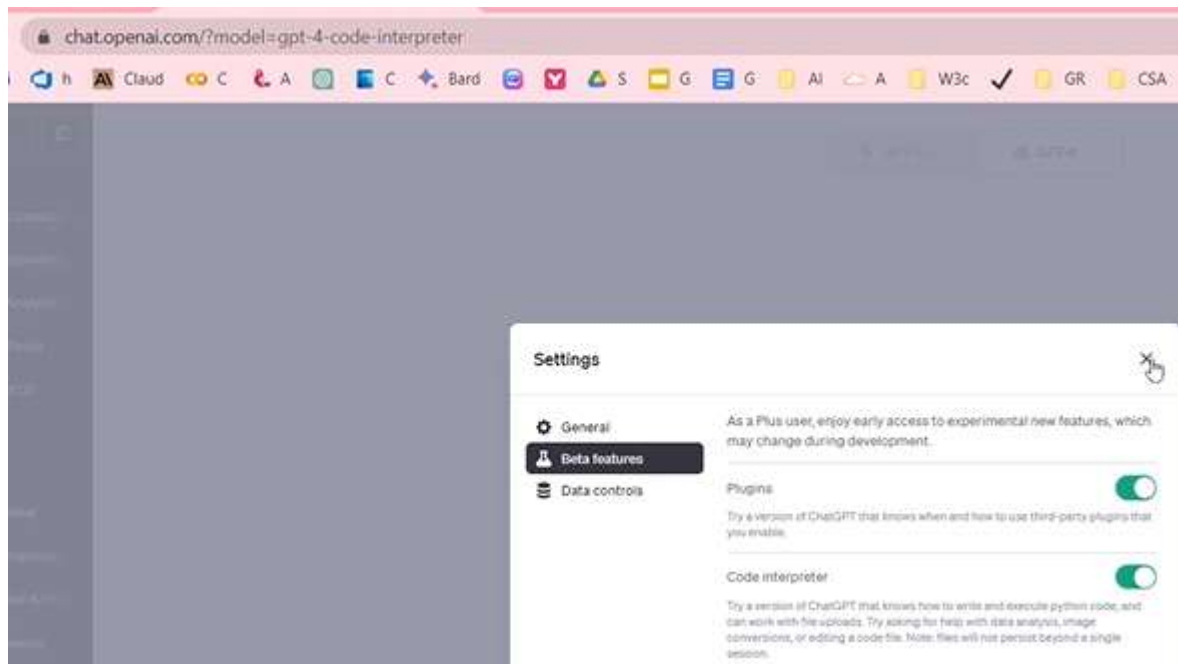
2: Prompt Injection
<https://chat.openai.com/share/203c637c-d80d-488e-b97d-d131f94ae494>

3: AWS IAM Policy Review
<https://chat.openai.com/share/2a0eb9d8-1ff1-4004-9e91-c506b16ae02d>

4: ChatGPT Code Interpreter use case
https://data.world/euclid46/privacy-security-technology/workspace/file?filename=Health+Data+Breaches_Expanded.xlsx

docs.google.com/presentation/d/1SZtEPFxtwq_V939fG2wTBmZ-mAxegv6/edit#slide=id.p3

Perplexity AI AI Security LLM G doc docs EC Bard



Preparing the Dataset

- ☐ Loading the dataset using the  Datasets library.
- ☐ Example: Loading the Wikitext 2 dataset.

Tokenization Using a Pre-trained Tokenizer

- ☐ Using AutoTokenizer class to download a pre-trained tokenizer.
- ☐ Tokenizing the text data.

Grouping Texts and Instantiating the Model

- ❑ Concatenating texts and splitting them into examples of a certain sequence length.
- ❑ Using the map method to split text into smaller chunks.
- ❑ Instantiating the model for fine-tuning.
- ❑ Using AutoModelForCausalLM to load the model checkpoint.

Setting up Training Arguments for Training

- ❑ Defining the training arguments, including learning rate, weight decay, and output directory.
- ❑ Setting up the Trainer for fine-tuning.
- ❑ Instantiating the Trainer class with the model, training arguments, and training/validation datasets.
- ❑ Training the model.
- ❑ Calling the training method of the Trainer to start the training process.

Evaluate

- ❑ Evaluating the fine-tuned model.
- ❑ Using the evaluation method of the Trainer to get the model's perplexity on the validation set.
- ❑ To cancel Colab Pro, please use:
 - ❑ pay.google.com
 - ❑ under cancel contracts for Colab

Using Jupyter Notebook, Pytorch, and Google Cobab

- ❑ Jupyter Notebook: A web-based interactive computational environment that allows creating and sharing documents containing live code, equations, visualizations, and narrative text.
- ❑ PyTorch: An open-source machine learning framework that provides a flexible and efficient platform for building and training neural networks, widely used for deep learning tasks.

The screenshot shows a Google Colab notebook interface. The browser address bar displays the URL: `colab.research.google.com/drive/1AjlmyvmU86ku5Gc0nbvOjL8KmaBmuJfs#scrollTo=DfJEnO4dmUCz`. The notebook title is "Section5-Fine-tune a language model". The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with icons for code and text. The main content area contains text instructions and a code cell. The code cell contains the following Python code:

```
from huggingface_hub import notebook_login
notebook_login()
```

Below the code cell, there are status messages: "Token is valid (permission: read)", "Your token has been saved in your configured git cre...", "Your token has been saved to /root/.cache/hug...", and "Login successful".

The "Notebook settings" dialog box is open, showing the following configuration:

- Runtime type: Python 3
- Hardware accelerator: GPU
- GPU type: A100
- Runtime shape: High-RAM
- ☐ Automatically run the first cell or section
- ☐ Omit code cell output when saving this notebook

The dialog box has "Cancel" and "Save" buttons at the bottom right.

colab.research.google.com/drive/1AjImyvU86ku5Gc0nbvOjLBKmaBmuJfs#scrollTo=IS2m25YM3I-z

Perplexity AI AI Security LLM G Y doc docs EC YouTube Twitter Bard

Section5-Fine-tune a language model

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

[23] Downloading (...)../main/vocab.json: 100% 1.04M/1.04M [00:00<00:00, 4.03MB/s]
 Downloading (...)../main/merges.txt: 100% 456k/456k [00:00<00:00, 2.36MB/s]
 Downloading (...)../main/tokenizer.json: 100% 1.36M/1.36M [00:00<00:00, 4.23MB/s]

We can now use the `map` method from the Datasets library to tokenize our text

```
def tokenize_function(examples):
    return tokenizer(examples["text"])
```

To optimize the preprocessing step, we utilize the "batched=True" parameter and employ four processes to handle the splits in our datasets object simultaneously. This approach helps expedite the processing. Once this step is completed, we no longer require the text column, so we discard it.

```
tokenized_datasets = datasets.map(tokenize_function, batched=True, num_proc=4, remove_columns=["text"])
```

WARNING:datasets.arrow_dataset:Loading cached processed dataset at /root/.cache/huggingface/datasets/wikitext/wikitext-2-raw-v1/1.
 WARNING:datasets.arrow_dataset:Loading cached processed dataset at /root/.cache/huggingface/datasets/wikitext/wikitext-2-raw-v1/1.
 WARNING:datasets.arrow_dataset:Loading cached processed dataset at /root/.cache/huggingface/datasets/wikitext/wikitext-2-raw-v1/1.

If we now look at an element of our datasets, we will see the text have been replaced by the "input_ids". the "input_ids" refer to the tokenized representation of the text in the dataset element. The input_ids are the numerical representations of the tokens that the model will use as input during training or inference. Additionally, the "attention_mask" field, also present in the dictionary, is a mask that indicates which tokens the

What Have You Learned?

- ☐ What is GenAI?
- ☐ What is LLM and the Transformer Architecture?
- ☐ Applications of GenAI
- ☐ LLM Application Architecture Patterns and Security Considerations
- ☐ LLM Application Tech Stacks and LLM Ops
- ☐ What is Prompt Engineering and its use in Cyber Security?
- ☐ How to do Finetune?

GenAI and LLM Impact to Cyber Security Professionals

- ☐ Learn GenAI and LLM
- ☐ Understand how to use GenAI and LLM for Cyber Security.
- ☐ Understand common security and privacy risks of LLM applications.
- ☐ Integrate with the LLM application development team.
- ☐ Bonus: Understand Secure LLM Ops.

Top GenAI Security Tools to Watch

- ☐ Google Cloud: Duet AI enhances threat visibility and response and simplifies security tasks.
- ☐ Microsoft: Security Copilot accelerates incident response and transforms security operations.
- ☐ Airgap Networks: ThreatGPT safeguards OT environments from cyber threats with AI/ML.
- ☐ Cisco: Generative AI in Collaboration and Security portfolios improves productivity.
- ☐ SentinelOne: AI-powered platform detects and remediates cyber attacks autonomously.
- ☐ <https://llmshield.com/faqs>
- ☐ <https://app.chaingpt.org/#chatbot>

app.chaingpt.org/#chatbot

Perplexity AI AI Security LLM G V doc docs EC Bard

Select a ChainGPT AI Model



Ask ChainGPT AI Your Questions

Ask ChainGPT AI any question related to Blockchain and Crypto. The AI can answer general and technical questions.

[Try Now](#) >



AI Solidity Smart Contract Generator

Create Smart Contracts! Describe the Solidity Smart Contract you wish to generate, and ChainGPT AI will write it for you.

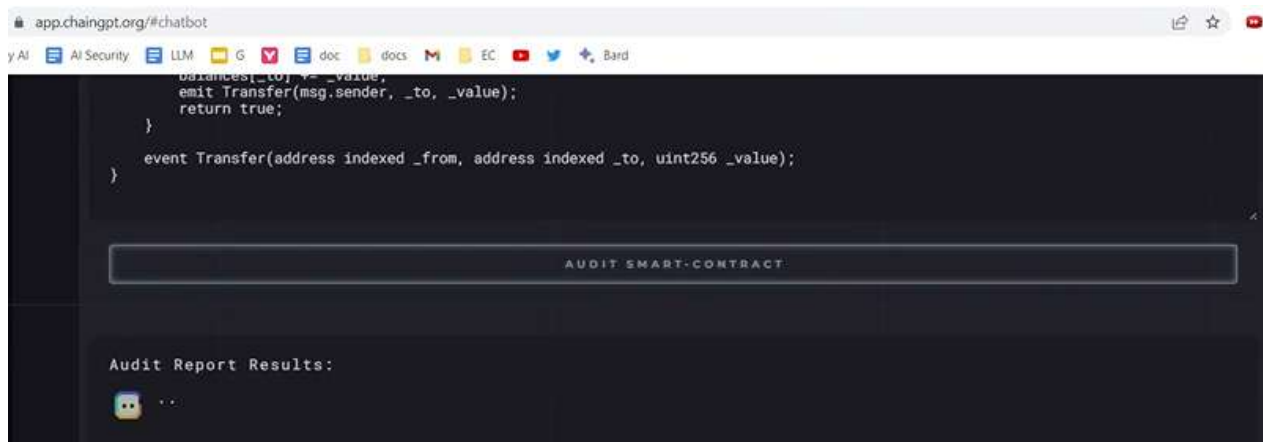
[Try Now](#) >



AI Solidity Smart Contract Auditor

Find vulnerabilities and exploits in Solidity Smart Contracts. Then, paste the Contract, click "audit," and our AI will do the rest.

[Try Now](#) >



Prompt engineering is the technique used to direct and constrain an LLM's behavior at inference time by providing clear instructions, role definitions, formatting requirements, and constraints in the prompt.

Prompt engineering Controls responses through instructions, examples, tone, format, and constraints.

Reinforcement learning Can shape behavior during training (e.g., RLHF), but it's not the usual runtime control technique in a quiz like this.

Sandboxing Isolates execution environments; it doesn't tell the model how to answer.

Regression testing Checks behavior changes over time; it doesn't control behavior directly.

LoRA (Low-Rank Adaptation) and **QLoRA (Quantized LoRA)** are techniques designed to make LLM fine-tuning much faster and more resource-efficient by training only a small number of additional parameters instead of updating the entire model.

Why not the others?

- **✗ Training the model longer** — increases training time rather than accelerating it.
- **✗ Increasing model complexity** — generally requires more computation.
- **✗ Minimizing training data volume** — may reduce training time, but can hurt performance and is not a specialized fine-tuning acceleration technique.

A key way to reduce bias in an LLM is to train it on **balanced and diverse datasets** that represent a wide range of perspectives, demographics, and contexts. This helps prevent the model from overrepresenting or underrepresenting particular groups or viewpoints.

Why not the others?

- **✗ Increasing model complexity** — does not inherently reduce bias.

- **✗ Expanding compute resources** — provides more training capacity but does not address bias directly.
- **✗ Utilizing adversarial networks** — can sometimes help in fairness research, but it is not the primary or most direct answer here.

Exercise 01: Examine and Illustrate Data Trends Using ChatGPT

- * On the search bar type <https://chat.openai.com> and press **Enter**.
- * Click on **Log in** to log in to your account.
- * After successful login, the homepage of ChatGPT appears. Switch to **GPT-4**. You will need a subscription account to access **GPT-4**.
- * Now we will download a file we will use in ChatGPT for code interpretation. Open a new tab. On the search bar, type https://data.world/euclid46/privacy-security-technology/workspace/file?filename=Health+Data+Breachess_Expanded.xlsx and press **Enter**
- * A login page appears. Sign in to the website to access the file.
- * After successful sign-in, the homepage of the website appears.
- * On the search bar, type [Health Data Breachess_Expanded.xlsx](#) and press **Enter**.
- * Click on the file to access it.
- * Now click on **Download** and select **Download (1.09 MB)** to download the file into the system. The file will be downloaded in the **Downloads** folder. The file is already saved in **Downloads** folder.
- * Open **File Explorer** from the taskbar and navigate to the **Downloads** folder. You can see that the file has been saved here.
- * Click on the file to open it. As you can see the file has different tabs by year of breach, by state, multiple incidents, and more.
- * To know what exactly the file contains, we will ask ChatGPT to analyze it. Minimize the window. Navigate back to the browser and switch to the ChatGPT tab.
- * Click on the attach file icon.
- * An **Open** window appears. Navigate to **Downloads** folder and select the **Health Data Breachess_Expanded**. Click on **Open**.
- * After the file is uploaded, click on the arrow icon to submit the file.
- * We get a response from ChatGPT asking how it can assist with the file. The response might vary.
- * Now type Please let me know what specific analysis or operations that you can do and press **Enter**.
- * ChatGPT will give a list of analyses and operations it can perform on the data. The response might vary.
- * Now type please do descriptive statistics and press **Enter**.
- * ChatGPT will perform descriptive statistics and display the output. The response might vary.
- * Now type can you generate graph for these and press **Enter**. If it asks for what type of graph, type **all** and press **Enter**.
- * As you can see ChatGPT has created a graph based on its analysis. The response might vary. If the response is not as per your prompt, click on the regenerate icon to get a different response.
- * Now type can you do Time Series Analysis and press **Enter**.
- * ChatGPT displays a line graph.
- * Now type can you do text analysis and press **Enter**.

- * ChatGPT creates a word cloud from the descriptions of the breaches reported in the dataset.
- * Now type please do Geographical Analysis and press **Enter**.
- * ChatGPT gives a response as shown below.
- * Next type can you show me it in table format and press **Enter**.
- * ChatGPT displays the dataset in table format.

Exercise 02: Executing and Evaluating Model Training and Data Preparation in Google Colab

- * On the search bar type <https://colab.research.google.com/drive/1AjJmyvmU86ku5Gc0nbvOjLBKmaBmuJfs?usp=sharing>
- * Close the **Executions** tab by clicking on the **X** icon.
- * Run the first code by clicking on the **Run cell** icon. It will ask you to sign in to your Google account. Sign in to your account by clicking on **Sign in** or create a new one if you don't already have an account.
- * After signing in to the account, you can see that it asks for a token. To get the token, click on the link **your Hugging Face tokens page**.
- * A new page opens. Here we must log in to our **Hugging Face** account. Enter your registered email address and password and click on **Login**.
- * We get redirected to the tokens page. Here click on **New token**.
- * A pop-up appears. In the **Name** field, type testing and click on **Generate a token**.
- * A token will be generated. Copy the token by clicking on the **Copy** icon.
- * Navigate back to the first tab.
- * Now paste the copied token in the **Token** field and click on **Login**.
- * The login is successful.
- * Now we have to install **Git-LFS**. Click on the **Run cell** icon to install.
- * After installation run the code to import transformers. You can see the version of transformers imported.
- * Scroll down to the **Preparing the dataset** section. Click on the **Run cell** icon to install the datasets.
- * Scroll down a little and here you can inspect your data.
- * Replace the **1** in the code with **10** and then **11** and run the code respectively. The code should now look like `datasets["train"][10]` and `datasets["train"][11]`. Observe the outputs.
- * Replace the **11** in the code with **1** run the code as shown below. This way you can inspect the data.
- * You can also see random data by running the code shown below. Run the code.
- * Now run the next code. You can see that it has downloaded a random wiki text.
- * Run the code again. You can see that we get a different output.
- * Scroll down to **Causal Language modeling**.
- * Here in the first code, **distilgpt2** model is being used. Run the code.
- * Now run the next code to import the token for this model.
- * Run the next code to use the map method to tokenize the text.
- * Run the next code. Observe that it says that **the text have been replaced by the "input_ids"**.
- * Run the next code to view the input_ids corresponding to different texts.
- * Observe that the **attention_mask** is **1**. An attention mask is essentially a matrix that specifies

which tokens in the input sequence should be attended to and which ones should be ignored during the computation of attention scores.

- * Replace the **100** in the code with **2** as shown below and run the code.
- * We do not get any output.
- * Now replace the **2** with **3** and run the code. You can see that we are getting a bigger output.
- * Run the next code to combine the text into one sequence and split it into chunks of block size 128.
- * Run the next code to group all the texts into one sequence.
- * Run the next code to split text into chunks using the map method.
- * Run the next code to decode the text.
- * Now we will run the code to import the **AutoModelForCausalLM** from transformers.
- * Run the next codes to install and import the dependencies as shown below.
- * In the code shown below we will get an error.
- * To fix the error first scroll to the top. Click on **Runtime** and select **Restart session**. A pop-up will appear, click on **Yes**.
- * Now run the first code. Enter the token and click on **Login**. You can view the token by navigating to the **Hugging Face** tab.
- * Run all the codes once again. As you can see now the code has been executed.
- * Run the next code to set up the model.
- * Now run the next code to train the model. It will take some time to generate the output.
- * You can see that the output is generated. It has completed 7,002 iterations. It also contains training loss and validation loss. Training loss measures how well the model fits the training data. Validation loss measures how well the model generalizes to unseen data.
- * Run the next code to evaluate the model and get its perplexity on the validation set. As you can see here perplexity is **38.17**. Perplexity evaluates the performance of language models.

.....

Client-side encryption helps maintain the confidentiality of LLM data in the cloud by encrypting the data before it leaves the user's environment. This reduces the risk of unauthorized access to sensitive information.

Why not the others?

- **✗ Disabling access controls** — weakens security.
- **✗ Permitting universal access** — exposes data to unauthorized users.
- **✗ Allowing plaintext storage** — leaves data vulnerable if storage is compromised.

In modern LLMs, **attention mechanisms** are the key component that enables the model to process and understand sequential text by focusing on the most relevant parts of the input when interpreting or generating each token.

Why not the others?

- **Embeddings** convert tokens into numerical vectors but do not model relationships across a sequence by themselves.

- **Convolutional layers** are mainly associated with CNNs and are not the core mechanism in LLMs.
- **Recurrent neural networks (RNNs)** were used for sequential data before transformers, but modern LLMs are based on attention mechanisms rather than RNNs.

Fine-tuning mitigates risks by tailoring a pre-trained LLM to a specific domain, task, or set of requirements. This can improve accuracy, relevance, compliance, and alignment with organizational policies, reducing the likelihood of inappropriate or incorrect outputs.

Why not the others?

- **✗ Enhancing computational efficiency** — not the primary purpose of fine-tuning.
- **✗ Accelerating software patching** — unrelated to LLM fine-tuning.
- **✗ Speeding up inference** — fine-tuning generally does not make the model significantly faster.

Ongoing security testing is important because LLM applications face evolving threats, attack techniques, and deployment environments. Regular testing helps identify **new and emerging vulnerabilities** before they can be exploited.

Why not the others?

- **✗ To improve usability** — useful, but not the primary security purpose.
- **✗ To detect zero-day exploits** — possible in some cases, but security testing cannot reliably detect all zero-days.
- **✗ To evaluate new features** — part of software testing, not specifically security testing.

Word embeddings convert words (or tokens) into numerical vector representations that machine learning models can process. These vectors capture semantic relationships between words, allowing the model to work with text mathematically.

Why not the others?

- **✗ Allowing unsupervised learning** — embeddings can be learned in various ways, but this is not their primary function.
- **✗ Facilitating attention** — attention mechanisms operate on embeddings, but embeddings themselves do not provide attention.
- **✗ Providing contextual understanding** — contextual understanding is primarily achieved through transformer layers and attention mechanisms; basic embeddings mainly represent words as vectors.

Model explainability techniques help developers understand how an LLM makes decisions and which data patterns influence its outputs. This can reveal potential biases in the training data and model behavior, making it easier to identify and address fairness issues.

Why not the others?

- **✗ Reinforcement learning** — a training method, not primarily used to evaluate bias.
- **✗ Lossless compression** — a data storage technique unrelated to bias analysis.
- **✗ Adversarial attacks** — mainly used to test robustness and security, not to directly evaluate training-data bias.

Anomaly detection helps identify unusual patterns of model usage that may indicate a **model extraction attack**, where an attacker repeatedly queries an LLM to reconstruct or approximate the model's behavior.

Examples of suspicious activity include:

- Excessive querying
- Automated probing patterns
- Unusual request volumes
- Attempts to systematically map model responses

Why not the others?

- **✗ Versioning models** — helps manage model updates, not detect attacks.
- **✗ Immutable infrastructure** — improves system integrity but does not detect extraction attempts.
- **✗ Manual code reviews** — useful for finding code issues, but not for detecting active model extraction attacks.

Prompt engineering is the primary technique used to directly control LLM behavior at inference time by shaping inputs (instructions, constraints, examples, and formatting rules) to guide the model's outputs.

Why not the others?

- **✗ Sandboxing** — isolates execution environments but does not control model responses.
- **✗ Regression testing** — evaluates changes over time but does not control behavior directly.
- **✗ Reinforcement learning** — influences behavior during training, not immediate control during use.

Continuous monitoring is important when operating LLMs because it helps identify **security incidents in real time**, such as:

- Prompt injection attacks
- Data leakage attempts
- Abnormal usage patterns
- Model misuse or abuse

While monitoring can also support debugging, uptime tracking, and performance evaluation, the key security-driven reason is early detection of threats.

A key drawback of **proprietary (closed-source) LLMs** is that users do not have access to the internal details of the model, such as:

- Training data sources
- Model architecture specifics
- Weights and parameters
- Decision-making process

This limits transparency, auditability, and explainability.

Why not the others?

- **✗ Mandatory hardware upgrades** — not typically required since models are cloud-hosted.
- **✗ Permissive licensing terms** — usually the opposite; licenses are restrictive.
- **✗ Unrestricted usage rights** — closed-source models generally have usage limits and restrictions.

1) Какво представляват LLM моделите (основи)

💡 Как работят

- LLM = модел, който предсказва следващия токен
- Работи с **токени** → **числа (embeddings)** → **вероятности**
- Не „разбира“ като човек, а изчислява най-вероятен текст

💡 Основна архитектура

- **Transformer** = основата на всички съвременни LLM-и
- използва **attention mechanisms**, които:
 - намират важните части от текста
 - позволяват контекст и дълги зависимости

2) Как се тренират LLM модели

⚡ Pipeline (много важно)

1. Събиране на данни (datasets)
2. Почистване и подготовка
3. Tokenization
 - текст $\rightarrow$ input_ids + attention_mask
4. Chunking (разбиване на текст)
5. Training (fine-tuning или pretraining)
6. Evaluation

⚡ Метрики

- **Loss** $\rightarrow$ колко грешки моделът
- **Validation loss** $\rightarrow$ обобщаване към нови данни
- **Perplexity** $\rightarrow$ качество на езиковия модел (по-ниско = по-добре)

3) Видове модели (важно за тестове)

⚡ LLM (GPT тип)

- генерира текст

⚡ Embedding модели (Sentence Transformers)

- превръщат текст във вектори
- мерят **семантична близост**

📖 пример:

- “happy person” $\approx$ “very happy person”
- но $\neq$ “sunny day”

4) Hugging Face екосистема

⚡ Какво има там:

- Models
- Datasets
- Tasks
- Licenses
- Libraries (Transformers)

⚡ Лицензи:

- **MIT / Apache 2.0** → най-свободни
- **closed models** → ограничен достъп

⚡ Open-source не означава:

- без ограничения
- или без лиценз

! 5) Cloud и модели (инфраструктура)

⚡ Google Vertex AI

- train + deploy ML модели
- за NLP, vision, fraud detection

⚡ Cloud LLMs

- намаляват инфраструктурните разходи
- но дават по-малка прозрачност

🔒 6) Security за LLM (много важно за курса)

⚡ Основни рискове:

- data poisoning (замърсяване на данни)
- model extraction attacks
- prompt injection
- bias в данните

⚡ Защити:

- **Anomaly detection** → засича атаки и подозрително поведение
- **Guardrails** → контрол на изхода на модела
- **Access controls** → задължителни
- **Client-side encryption** → защита на чувствителни данни
- **Continuous monitoring** → засича инциденти



7) Fine-tuning и адаптация

◆ Какво е fine-tuning

- адаптира предварително обучен модел към конкретна задача

◆ Ползи:

- по-добра точност в конкретна област
- намалява риск от грешки в domain

◆ Ускорение:

- LoRA / QLoRA → по-бързо и евтино обучение
-



8) Control на LLM поведение

◆ Основни техники:

- **Prompt engineering** → най-важното за контрол на изхода
 - **Reinforcement learning (RLHF)** → обучение чрез човешка обратна връзка
 - **Regression testing** → проверка на промени
 - **Sandboxing** → изолация (не контрол на отговорите)
-



9) Data analysis с AI

◆ ChatGPT като аналитичен инструмент:

- descriptive statistics
- графики
- time series analysis
- text analysis (word clouds)
- geographical analysis

☞ Извод:

LLM може да е **data analyst асистент**

10) Реален ML pipeline (Google Colab)

⚡ Какво се прави:

- логване в Hugging Face
- зареждане на dataset
- tokenization
- training (GPT-2 / distilgpt2)
- evaluation

⚡ Важно:

- моделите работят с числа, не с текст
- attention mask контролира важноста на токениите

11) Основни концепции за тестове (най-важното)

- ✓ Transformer = основна архитектура
- ✓ Attention = контекст
- ✓ RLHF = човешка обратна връзка
- ✓ Guardrails = контрол на output
- ✓ Anomaly detection = security monitoring
- ✓ Client-side encryption = защита на данни
- ✓ LoRA = ускорено fine-tuning
- ✓ Embeddings = текст → вектори
- ✓ Perplexity = качество на модел
- ✓ Open-source ≠ без лиценз
- ✓ LLM ≠ AGI



Ако трябва да го обобщя в 1 изречение:

📖 Курсът ти учи как работят LLM моделите, как се тренират и използват, как се анализират данни с AI и най-важното — как се защитават AI системи от атаки и злоупотреби.



Flashcards – LLM, AI & Cybersecurity



Основи на LLM

Q: Какво е LLM?

A: Модел, който предсказва следващия токен в текст.

Q: Каква архитектура използват съвременните LLM?

A: Transformer.

Q: Какво прави attention механизмът?

A: Позволява на модела да се фокусира върху важните части от текста.

Q: LLM има ли истинско разбиране като човек?

A: Не, работи чрез вероятности и статистика.



Видове модели

Q: Какво са embedding модели?

A: Модели, които превръщат текст във вектори.

Q: За какво се използват sentence transformers?

A: За измерване на семантична сходност между текстове.

Q: Embeddings показват ли значение или точни думи?

A: Значение (семантика), не точен текст.



Обучение на модели

Q: Какво е tokenization?

A: Превръщане на текст в числа (tokens).

Q: Какво е input_ids?

A: Числово представяне на текста.

Q: Какво е attention mask?

A: Показва кои токени да се вземат предвид.

Q: Какво е loss?

A: Грешката на модела.

Q: Какво е perplexity?

A: Мярка за качеството на езиков модел (по-ниско = по-добре).



Fine-tuning & обучение

Q: Какво е fine-tuning?

A: Адаптиране на предварително обучен модел към конкретна задача.

Q: Какво е LoRA / QLoRA?

A: Метод за по-бързо и евтино fine-tuning.

Q: Как fine-tuning помага за риск?

A: Адаптира модела към конкретен домейн.



Hugging Face

Q: Какво е Hugging Face?

A: Платформа за модели, datasets и AI инструменти.

Q: Какво означава Apache-2.0 лиценз?

A: Свободен лиценз за използване и модификация.

Q: Open-source означава ли без ограничения?

A: Не, винаги има лицензни условия.

Q: Falcon 40B какъв модел е?

A: Голям open-source LLM с 40 милиарда параметъра.



AI инфраструктура

Q: Какво е Google Colab?

A: Облачна среда за ML с GPU.

Q: Какво прави Hugging Face token?

A: Дава достъп до модели и datasets.

Q: Какво е Transformers библиотека?

A: Основна библиотека за LLM модели.



Data analysis с AI

Q: Какво е descriptive statistics?

A: Основни статистически показатели (средно, мин/макс и т.н.).

Q: Какво е time series analysis?

A: Анализ на данни във времето.

Q: Какво е word cloud?

A: Визуализация на най-чести думи в текст.

Q: Какво показва географски анализ?

A: Разпределение на данни по локации.

Cybersecurity за LLM

Q: Какво е data poisoning?

A: Замърсяване на тренировъчните данни с грешна информация.

Q: Какво е model extraction attack?

A: Кражба на модел чрез много заявки.

Q: Какво е prompt injection?

A: Манипулиране на входа, за да се промени поведението на модела.

Q: Какво са guardrails?

A: Правила, които контролират изхода на LLM.

Q: Какво прави anomaly detection?

A: Засича подозрително поведение или атаки.

Q: Какво е client-side encryption?

A: Криптиране на данни преди да се изпратят в облака.

Q: Какво прави access control?

A: Ограничава кой има достъп до системата.



Контрол на LLM

Q: Какво е prompt engineering?

A: Контролиране на модела чрез добре формулирани инструкции.

Q: RLHF какво е?

A: Обучение на модел чрез човешка обратна връзка.

Q: Sandboxing контролира ли отговорите на модела?

A: Не, само изолира средата.



Performance & evaluation

Q: Как се оценява LLM?

A: Чрез loss и perplexity.

Q: По-ниска perplexity означава?

A: По-добър модел.



Важни концепции

Q: LLM = AGI?

A: Не.

Q: Open-source модели имат ли лицензи?

A: Да.

Q: AI може ли да генерира bias?

A: Да, ако данните са пристрастни.

⚡ CHEAT SHEET – Generative AI for Cybersecurity

LLM основи

- **LLM** = предсказва следващ токен (не „разбира“)
 - **Transformer** = основна архитектура
 - **Attention** = фокус върху важни части от текста
-

Видове модели

- **LLM (GPT)** → генерира текст
 - **Embeddings** → текст → вектори (семантика)
 - **Sentence Transformers** → сходство между текстове
-

Обучение (ML pipeline)

1. Data
 2. Tokenization (text → input_ids)
 3. Training
 4. Evaluation
- **Loss** = грешка
 - **Validation loss** = генерализация
 - **Perplexity** ↓ = по-добър модел
-

Fine-tuning

- адаптира модел към конкретна задача
 - **LoRA / QLoRA** = бързо и евтино fine-tuning
-

Hugging Face

- платформа за **models + datasets + tools**
 - **Apache-2.0 / MIT** = свободни лицензи
 - open-source ≠ без ограничения
-

AI анализ

- **Descriptive stats** = основна статистика
 - **Time series** = анализ във времето
 - **Text analysis** = word cloud
 - **Geographic analysis** = по локации
-

Cybersecurity за LLM

- **Data poisoning** = замърсени данни
- **Model extraction** = копиране на модел чрез заявки
- **Prompt injection** = манипулация на входа

Защити:

- **Guardrails** = контрол на output
 - **Anomaly detection** = засича атаки
 - **Access control** = кой има достъп
 - **Encryption (client-side)** = защита на данни
-

Control на LLM

- **Prompt engineering** = контрол чрез инструкции
 - **RLHF** = човешка обратна връзка
 - **Regression testing** = проверка на промени
 - **Sandboxing** = изолиране (не контрол)
-

Evaluation

- **Perplexity ↓** = по-добър модел
 - **Loss** = грешка на модела
-

Важни истини

- LLM ≠ AGI
- AI може да има bias
- Open-source ≠ без лиценз

- Attention = ключът към контекста
-

10-секунден финален спринт

- ☞ Transformer + Attention = LLM
 - ☞ Tokens + embeddings = как работи текстът
 - ☞ Fine-tuning = адаптация
 - ☞ Guardrails + anomaly detection = security
 - ☞ Perplexity = качество
-

2. Какво е Generative AI и LLM

◆ Generative AI

- AI, който генерира ново съдържание
 - текст
 - код
 - изображения

◆ LLM (Large Language Models)

- част от Generative AI
- работят чрез:
 - tokens
 - embeddings
 - вероятно прогнозиране

◆ Основна архитектура

- Transformer
 - Attention mechanism
-

3. Архитектура и tech stack (много важно за курса)

◆ Основни компоненти:

- Data (datasets)
- Models (LLMs)
- Training (PyTorch, Transformers)
- Inference (API / deployment)
- Cloud (Colab, Vertex AI)

◆ Key tools:

- Hugging Face
- Google Colab
- PyTorch
- Transformers library



4. Как работи ML pipeline

1. Data collection
2. Preprocessing
3. Tokenization
4. Training
5. Evaluation

◆ Метрики:

- Loss
- Validation loss
- Perplexity



5. Fine-tuning (ядро на практическата част)

◆ Какво е:

- адаптиране на LLM към конкретна задача

◆ Методи:

- Full fine-tuning
- LoRA / QLoRA (по-бързи и евтини)

◆ Използване:

- cybersecurity datasets
 - domain-specific AI
-

6. AI + Cybersecurity (най-важната част)

Основни рискове:

1. Data poisoning

- замърсяване на тренировъчни данни

2. Model extraction

- копиране на модел чрез заявки

3. Prompt injection

- манипулиране на входа

4. Bias

- изкривени резултати от данни
-

Защити:

- **Guardrails** → контрол на output
 - **Anomaly detection** → засича атаки
 - **Access control** → кой има достъп
 - **Encryption (client-side)** → защита на данни
 - **Continuous monitoring** → откриване на инциденти
-

7. Control на LLM поведения

◆ Техники:

- Prompt engineering → контрол чрез текст
- RLHF → човешка обратна връзка
- Regression testing → проверка на версии

- Sandboxing → изолиране
-

8. Open-source vs Closed-source LLM

Open-source:

- прозрачност
- контрол
- лицензи
- нужда от инфраструктура

Closed-source:

- лесно използване
 - по-малко инфраструктура
 - липса на прозрачност
 - vendor lock-in
-

9. Практически умения (hands-on част)

Ще можеш да:

- работиш с Hugging Face модели
 - използваш datasets
 - fine-tune LLM в Google Colab
 - анализираш резултати (loss, perplexity)
 - използваш AI за security анализ
-

10. Data & AI анализ

- descriptive statistics
 - time series analysis
 - text analysis (NLP)
 - geospatial analysis
 - visualization (graphs)
-

⚡ 11. Какво реално иска курсът от теб (най-важното)

☞ Да можеш да:

Разбираш

- как работят LLM-и
- как се тренират
- как се използват

Разпознаваш

- security рискове
- атаки към AI системи

Използваш

- Hugging Face
- Colab
- fine-tuning

Оценяваш

- open vs closed models
- рискове vs ползи

🎯 12. Единственото изречение за изпита

Generative AI се използва в cybersecurity чрез LLM-и, които се базират на transformer архитектура, но създават нови рискове като data poisoning и prompt injection, които се контролират чрез guardrails, anomaly detection и fine-tuning.

ML (Machine Learning) или **машинно обучение** е дял от изкуствения интелект, при който компютрите се учат да изпълняват задачи **чрез данни**, вместо да бъдат изрично програмирани с правила.



Какво означава това?

Вместо да пишеш:

„ако е X, направи Y“

ти даваш на модела:

- много данни (примери)
 - и той сам „намира закономерности“
-

Пример

Спам филтър:

- даваш имейли + етикети (spam / not spam)
 - моделът се учи да разпознава спам сам
-



Как работи ML (опростено)

1. Събиране на данни
 2. Обучение на модел
 3. Моделът намира модели (patterns)
 4. Прави прогнози върху нови данни
-

◆ Основни типове ML

1. Supervised learning

- има вход + правилен отговор
- пример: класификация на спам

2. Unsupervised learning

- няма етикети
- моделът групира данни сам

3. Reinforcement learning

- учене чрез награди и наказания



ML в контекста на твоя курс

LLM (като ChatGPT) са ML модели, които:

- учат от огромни текстови данни
- предсказват следващия token
- използват **deep learning + transformer** архитектура



Най-кратко:

ML е метод, при който компютърът се учи от данни да прави прогнози или решения без да бъде директно програмиран.

Ето ти ясната връзка между **AI vs ML vs DL**, така както най-често се иска на изпити:



AI vs ML vs DL (връзката)

1. Artificial Intelligence (AI)

Най-широкото понятие

AI = всяка система, която „имитира интелигентност“

Примери:

- чатботове
- системи за разпознаване на лица
- автономни автомобили
- LLM (ChatGPT)

✦ AI включва всичко:

- правила (rule-based systems)
 - ML
 - DL
-

2. Machine Learning (ML)

☞ **подмножество на AI**

ML = AI, който се учи от данни

Вместо правила → използва данни.

Как работи:

- даваш данни
- моделът намира patterns
- прави прогнози

Примери:

- spam detection
- recommendation systems
- fraud detection

3. Deep Learning (DL)

☞ **подмножество на ML**

DL = ML с много слоеве (neural networks)

Основа:

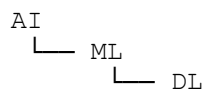
- изкуствени невронни мрежи

Използва се за:

- LLM (ChatGPT, GPT)
- image recognition
- speech recognition



Йерархия (много важно)





Къде са LLM моделите?

☞ LLM = Deep Learning

- базирани на **Transformer**
- използват **neural networks**
- обучават се с огромни данни

▮ Разлика накратко

Понятие	Какво е	Пример
AI	всичко „умно“	робот, chatbot
ML	учи от данни	spam filter
DL	невронни мрежи	ChatGPT

⚡ Най-важното за изпит

☞ запомни това:

- **AI** = общото поле
- **ML** = учене от данни
- **DL** = невронни мрежи
- **LLM** = DL + Transformer



10-секундна версия

AI е всичко, което имитира интелигентност.

ML е AI, който учи от данни.

DL е ML с невронни мрежи, който захранва LLM моделите.

